

Task 2 Documentation

Titanic Dataset Data Cleaning & Exploratory Data Analysis (EDA)

Introduction

Data Cleaning and Exploratory Data Analysis (EDA) are essential steps in every Data Science and Machine Learning project.

Raw datasets often contain:

- Missing values 
- Duplicate records 
- Outliers 
- Inconsistent formats 
- Noisy data 

If these problems are not handled properly, Machine Learning models may produce inaccurate predictions.

This project focuses on cleaning and analyzing the famous Titanic dataset using Python and various data science libraries.

The project demonstrates:

- ✓ Data preprocessing
 - ✓ Missing value handling
 - ✓ Duplicate removal
 - ✓ Outlier treatment
 - ✓ Visualization techniques
 - ✓ Correlation analysis
 - ✓ Feature encoding
 - ✓ Exporting cleaned data
-

Objective

The main objective of this project is to:

- Understand the Titanic dataset
 - Identify missing and inconsistent data
 - Clean the dataset using preprocessing techniques
 - Visualize important patterns and relationships
 - Prepare the dataset for Machine Learning applications
-

Dataset Description

The Titanic dataset contains information about passengers who traveled on the Titanic ship.

The dataset includes features related to:

- passenger details
 - ticket information
 - socioeconomic status
 - survival outcome
-

Dataset Features

Feature	Description
PassengerId	Unique ID assigned to each passenger
Survived	Survival status (0 = No, 1 = Yes)
Pclass	Passenger class
Name	Passenger name
Sex	Gender
Age	Passenger age
SibSp	Number of siblings/spouses aboard
Parch	Number of parents/children aboard
Ticket	Ticket number
Fare	Ticket fare
Cabin	Cabin number
Embarked	Port of embarkation

Technologies Used

Technology	Purpose
 Python	Programming Language
 Pandas	Data manipulation and analysis
 NumPy	Numerical operations
 Matplotlib	Data visualization
 Seaborn	Statistical visualization
 Scikit-learn	Missing value imputation

Libraries Imported

```
import pandas as pd
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt
import warnings

from sklearn.impute import SimpleImputer
```

Step 1: Loading the Dataset

The dataset is loaded using Pandas.

```
df = pd.read_csv("titanic dataset.csv")
```

After loading the dataset:

- first 5 rows are displayed
- dataset structure is checked
- data types are verified
- shape of dataset is examined

Example:

```
print(df.head())
print(df.info())
print(df.shape)
```

Step 2: Checking Missing Values

Missing values are common in real-world datasets.

The project checks missing values using:

```
df.isnull().sum()
```

This helps identify columns containing null values.

Missing value percentages are also calculated:

```
missing_percent = (df.isnull().sum() / len(df)) * 100
```

Step 3: Visualizing Missing Values

Visualizations make it easier to understand missing data patterns.

Two visualizations are created:

Missing Value Bar Plot

Displays the number of missing values in each column.

Missing Value Heatmap

Displays exact locations of missing values.

Code:

```
sns.heatmap(df_visual.isnull(), cbar=False)
```

Generated file:

```
missing_values_overview.png
```

Step 4: Handling Missing Values

Different imputation techniques are used depending on the data type.

Column	Missing Value Technique
--------	-------------------------

Column	Missing Value Technique
Age	Mean Imputation
Fare	Median Imputation
Embarked	Mode Imputation

Mean Imputation for Age

The average age is used to replace missing values.

```
age_imputer = SimpleImputer(strategy='mean')
df['Age'] = age_imputer.fit_transform(df[['Age']]).ravel()
```

Median Imputation for Fare

Median is used because fare values may contain outliers.

```
fare_imputer = SimpleImputer(strategy='median')
df['Fare'] = fare_imputer.fit_transform(df[['Fare']]).ravel()
```

Mode Imputation for Embarked

Most frequent value is used for categorical data.

```
embarked_imputer = SimpleImputer(strategy='most_frequent')
df['Embarked'] = embarked_imputer.fit_transform(df[['Embarked']]).ravel()
```

Step 5: Dropping Unnecessary Columns

The Cabin column contains too many missing values.

Therefore, it is removed from the dataset.

```
if 'Cabin' in df.columns:
    df.drop(columns=['Cabin'], inplace=True)
```

Step 6: Duplicate Removal

Duplicate rows are checked using:

```
df.duplicated().sum()
```

Duplicates are removed:

```
df.drop_duplicates(inplace=True)
```

Step 7: Outlier Detection

Outliers are abnormal values that differ significantly from other observations.

The Fare column is analyzed using a boxplot.

```
sns.boxplot(x=df[ 'Fare' ])
```

Generated file:

```
fare_boxplot_before.png
```

Step 8: Outlier Treatment Using IQR Method

The Interquartile Range (IQR) method is used.

Formula

Q1 = 25th percentile

Q3 = 75th percentile

$IQR = Q3 - Q1$

Lower and upper limits:

Lower Bound = $Q1 - 1.5 \times IQR$

Upper Bound = $Q3 + 1.5 \times IQR$

Code:

```
Q1 = df[ 'Fare' ].quantile(0.25)
```

```
Q3 = df[ 'Fare' ].quantile(0.75)
```

```
IQR = Q3 - Q1
```

Extreme values are capped.

Step 9: Distribution Analysis

Histograms are used to analyze data distributions.

Age Distribution

Displays how passenger ages are distributed.

Fare Distribution

Displays how ticket fares are distributed.

Generated file:

age_fare_distributions.png

Step 10: Survival Analysis

Survival patterns are analyzed using count plots and categorical plots.

Visualizations include:

- survival count comparison
- survival by gender
- survival by passenger class

Generated files:

survival_counts.png

survival_by_sex_pclass.png

Step 11: Correlation Analysis

Correlation analysis helps identify relationships between numeric variables.

A correlation heatmap is generated.

```
corr_matrix = df[numeric_cols].corr()
```

Generated file:

correlation_matrix.png

Step 12: Feature Encoding

Machine Learning models require numerical data.

Therefore categorical values are converted into numeric values.

Gender Encoding

Original	Encoded
male	0
female	1

Code:

```
df['Sex'] = df['Sex'].map({  
    'male': 0,  
    'female': 1  
})
```

Embarked Encoding

Original	Encoded
S	0
C	1
Q	2

Code:

```
df['Embarked'] = df['Embarked'].map({  
    'S': 0,  
    'C': 1,  
    'Q': 2  
})
```

Step 13: Saving Cleaned Dataset

The cleaned dataset is exported as a CSV file.

```
df.to_csv("cleaned_test.csv", index=False)
```

Generated file:

cleaned_test.csv

Output Files Generated

File Name	Description
missing_values_overview.png	Missing value visualization
fare_boxplot_before.png	Fare outlier boxplot
age_fare_distributions.png	Distribution plots
survival_counts.png	Survival count plot
survival_by_sex_pclass.png	Survival analysis
correlation_matrix.png	Correlation heatmap
cleaned_test.csv	Final cleaned dataset

Sample Visualizations

Missing Value Heatmap

Displays missing data patterns.

Survival Analysis

Shows how survival depends on:

- gender
- passenger class

Correlation Heatmap

Displays relationships among numeric variables.

Learning Outcomes

This project helped in understanding:

- ✓ Data Cleaning
 - ✓ Exploratory Data Analysis
 - ✓ Missing Value Treatment
 - ✓ Outlier Handling
 - ✓ Data Visualization
 - ✓ Feature Engineering
 - ✓ Dataset Preparation for Machine Learning
-

Future Improvements

Possible future enhancements:

-  Machine Learning Model Training
 -  Feature Scaling
 -  Hyperparameter Tuning
 -  Streamlit Dashboard
 -  Interactive Visualization
 -  Model Evaluation Metrics
-

Conclusion

This project successfully demonstrates a complete workflow for:

- cleaning raw data
- handling missing values
- detecting outliers
- performing exploratory data analysis
- visualizing insights
- preparing data for machine learning

The Titanic dataset was transformed from raw data into a clean and structured dataset ready for advanced analysis and predictive modeling  